

Lab 2 - Stateful Web Tracking

Web Privacy

Houda Toudali

UM6P - College of Computing

September 2026

1 Objective

The objective of this lab was to understand how a stateless HTTP service can recognize the same browser across several requests. The work focused on HTTP headers, cookies, third-party tracking, first-party analytics, and cookie syncing. Browser Developer Tools and Flask server logs were used to compare what is stored in the browser with what is received by the server.

2 Basic Flask Website and HTTP Headers

A first Flask application was created and served locally. The custom hostname `lab.test` was then mapped to the local machine.



Figure 1: First Flask page running locally.

The Network panel was used to inspect the HTTP exchange. The browser sent request headers such as `Host`, `User-Agent`, and `Accept`. The Flask application returned response headers, including the custom header `X-Lab-Message`. The same exchange could also be observed from the Flask terminal.

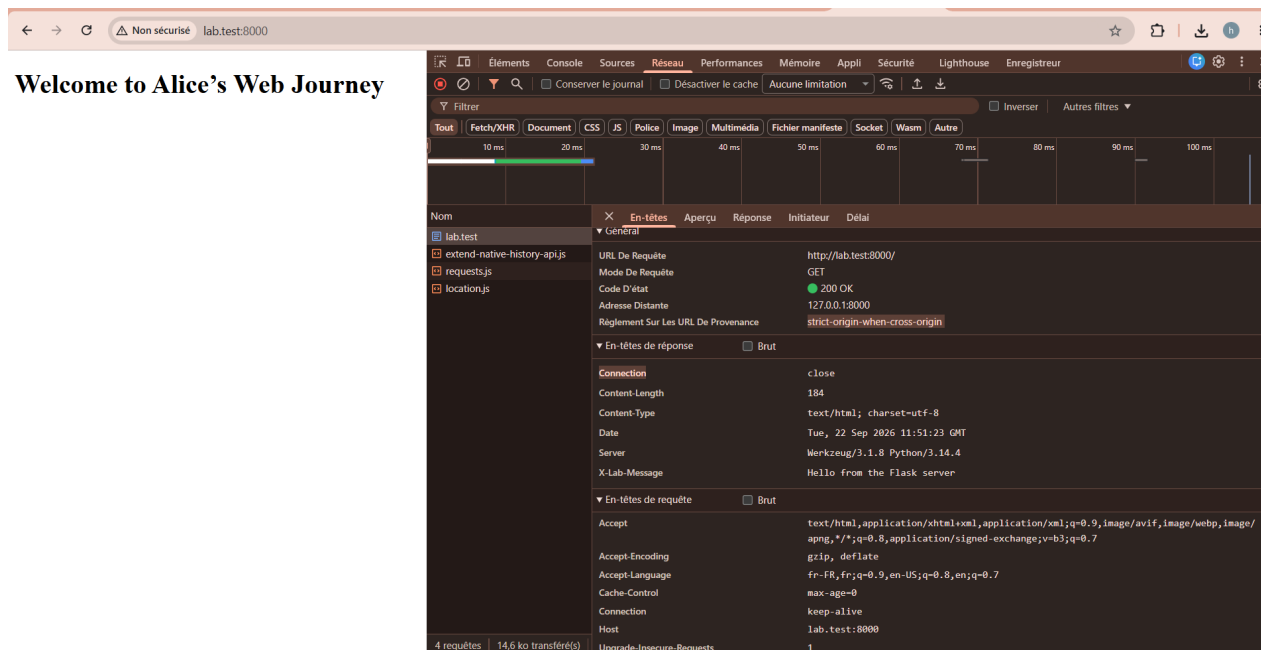


Figure 2: HTTP request and response headers in Developer Tools.

```
(venv) wwwto@houda:/mnt/c/Users/wwwto/Desktop/S3_CI2/Information Security/Labs/Lab 2/statefulTracking$ python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:8000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 497-920-084

--- HTTP REQUEST HEADERS ---
Host: lab.test:8000
Connection: keep-alive
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: fr-FR,fr;q=0.9,en-US;q=0.8,en;q=0.7

--- HTTP RESPONSE HEADERS ---
Content-Type: text/html; charset=utf-8
Content-Length: 184
X-Lab-Message: Hello from the Flask server
```

Figure 3: HTTP headers printed by the Flask server.

3 Stateful Tracking with Cookies

The server was modified to assign a random identifier called `aid`. On the first visit, the identifier was stored as a cookie in the browser. On later requests, the browser sent the same value back in the `Cookie` request header, allowing the server to recognize the browser.

Nom	Valeur	Domaine	Chemin ...	Expires /...	Taille	HttpOnly	Sécurisé	SameSite	Site de l...	Cross Site	Priorité
aid	6624241d3ce60a93	lab.test	/	Session	19						Medium

Figure 4: The aid cookie stored in the browser.

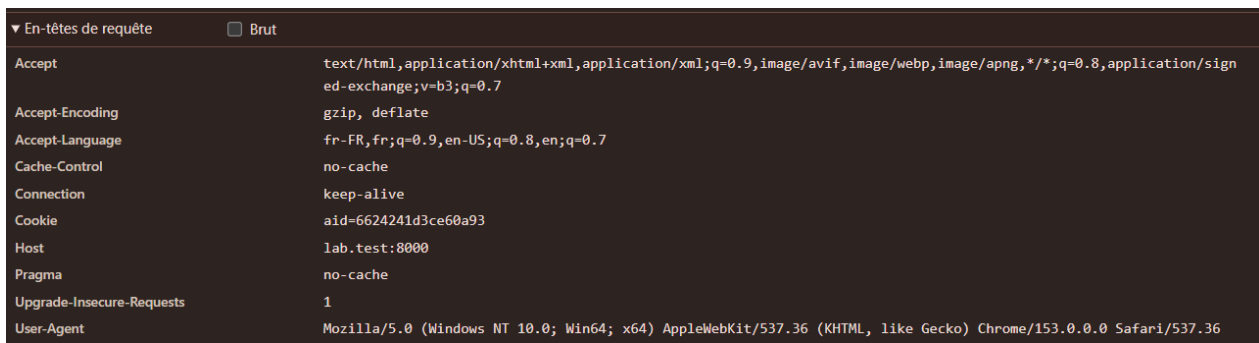


Figure 5: The browser sends the aid cookie in a later HTTP request.

After closing and reopening the browser, the same aid value was still visible in this browser configuration. Since the first version did not define an explicit expiration time, persistence was then tested more clearly with `Max-Age=3600`, which produced a visible expiration time.

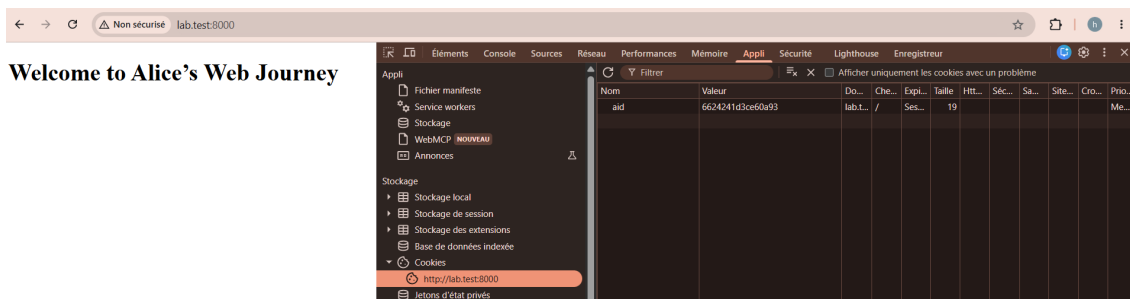


Figure 6: The same aid value observed after reopening the browser.

The cookie was also tested with different attributes. With `HttpOnly`, the cookie remained stored but was no longer accessible through `document.cookie`. A `SameSite=Lax` value was also tested to observe how cross-site behavior is controlled.

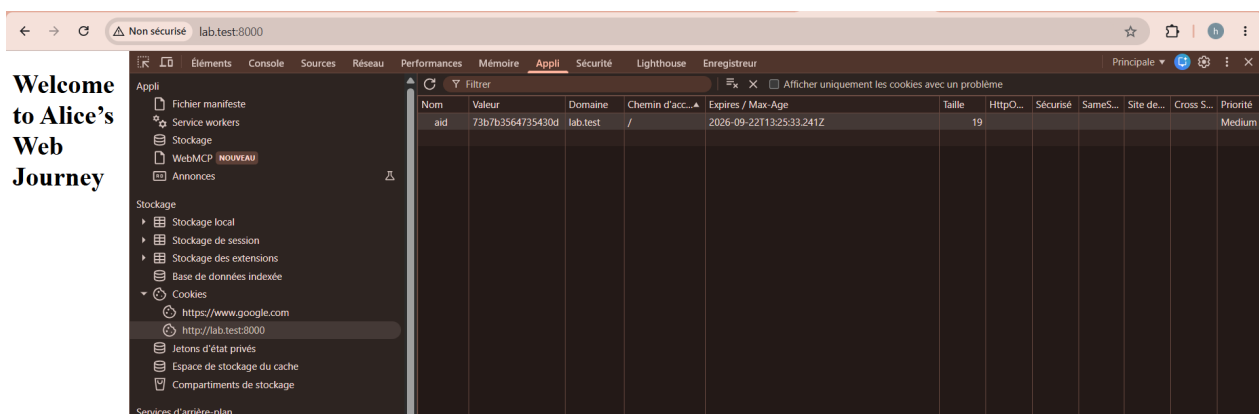


Figure 7: Persistent cookie created with an explicit Max-Age.

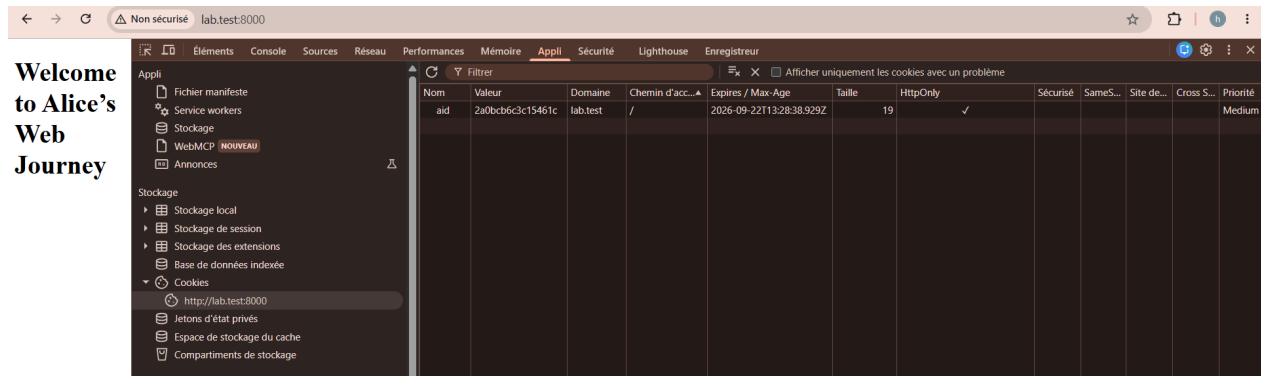


Figure 8: The HttpOnly attribute enabled for the cookie.

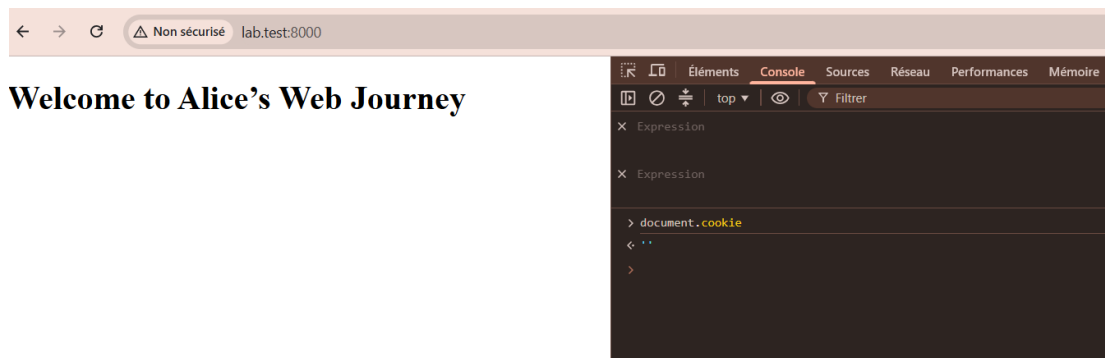
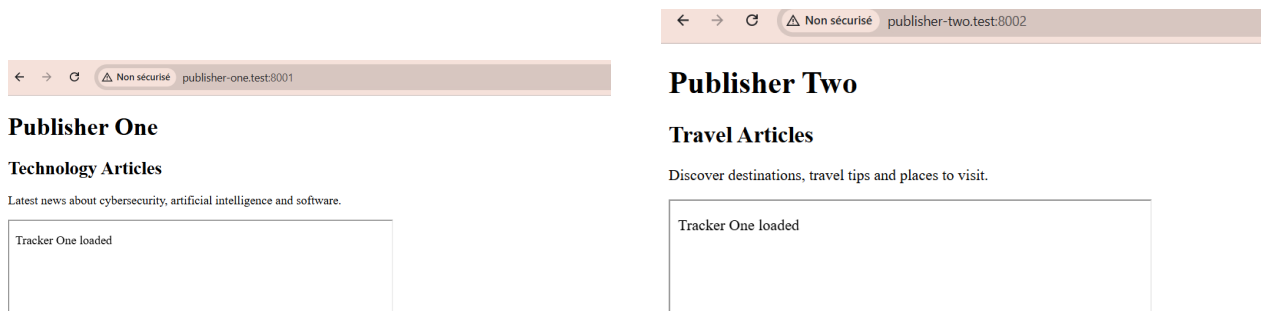


Figure 9: JavaScript cannot read the HttpOnly cookie.

4 Challenge 1 - Third-Party Tracking

Two independent publisher websites were created. Publisher One contained technology content and Publisher Two contained travel content. Both pages embedded the same tracker through an iframe.



Publisher One with Tracker One embedded.

Publisher Two with Tracker One embedded.

Figure 10: The same tracker is loaded by two different publishers.

At first, the tracker sent a **Set-Cookie** response but the browser did not reuse the cookie in the iframe context. This showed the effect of modern third-party cookie restrictions. The tracker cookie was then tested with **SameSite=None** and **Secure** over local HTTPS.

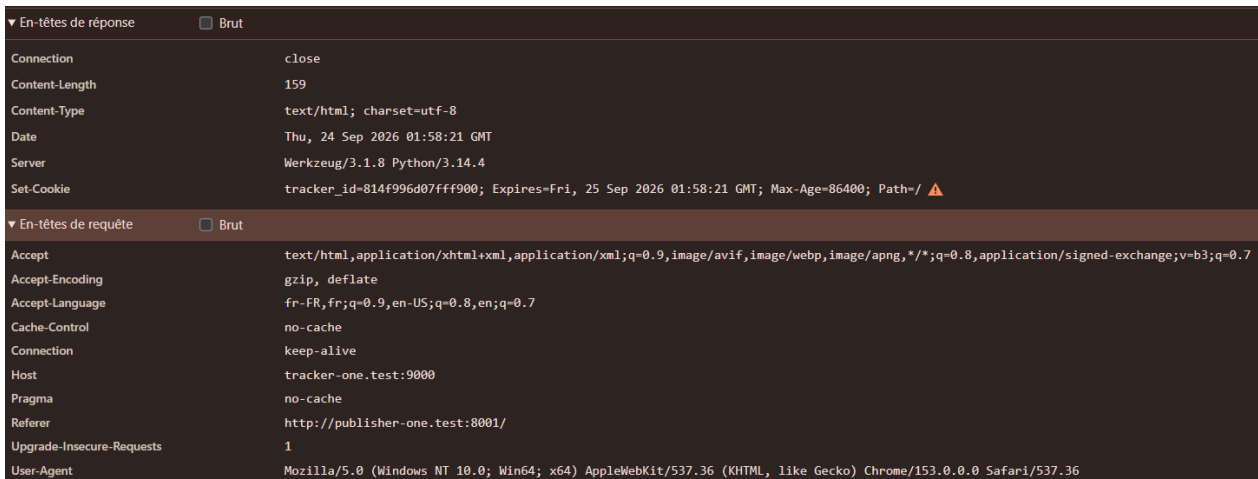


Figure 11: Tracker response containing Set-Cookie during the third-party test.

After the cookie settings were adapted, the same tracker identifier was reused for both publishers. The server reconstructed the visits in chronological order. The identifier `e4c3713d960bf2f3` appeared for both the technology and travel publisher visits.

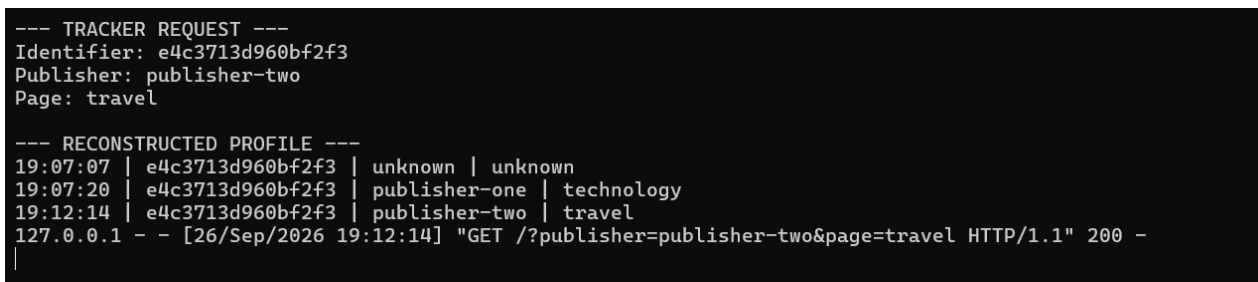


Figure 12: One tracker identifier linked to visits on both publishers.

The earlier `unknown` entry in the terminal came from opening the tracker directly while validating the local HTTPS certificate; the publisher entries are the tracking events used for the comparison. This demonstrates the privacy impact of third-party tracking: one external service can associate activity from different websites when the same identifier is available across those visits.

5 Challenge 2 - First-Party Analytics

The next experiment used a JavaScript file served by `analytics.test`. Although the script came from the analytics server, it executed inside the context of each publisher page. Therefore, `document.cookie` created a first-party cookie for the current publisher.

Publisher One and Publisher Two received different `analytics_id` values because cookies are isolated by domain.

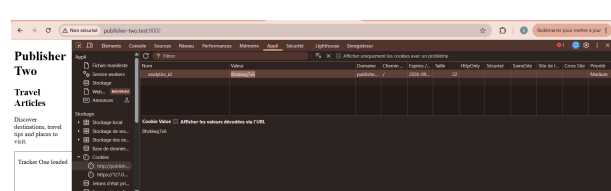
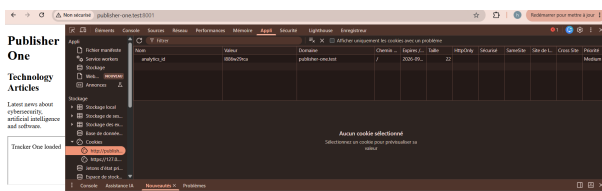


Figure 13: Different first-party analytics identifiers for the two publishers.

The request that downloaded `analytics.js` did not automatically contain the publisher's `analytics_id`. The cookie belongs to the publisher domain, not to `analytics.test`.



Figure 14: Request to the analytics server without the publisher's first-party cookie.

The script was then modified to explicitly send the identifier, publisher hostname, and page to an analytics collection endpoint. The analytics server received two different identifiers: `1886w29rca` for Publisher One and `8hskiwg7x6` for Publisher Two. As a result, it reconstructed two separate profiles instead of one combined profile.

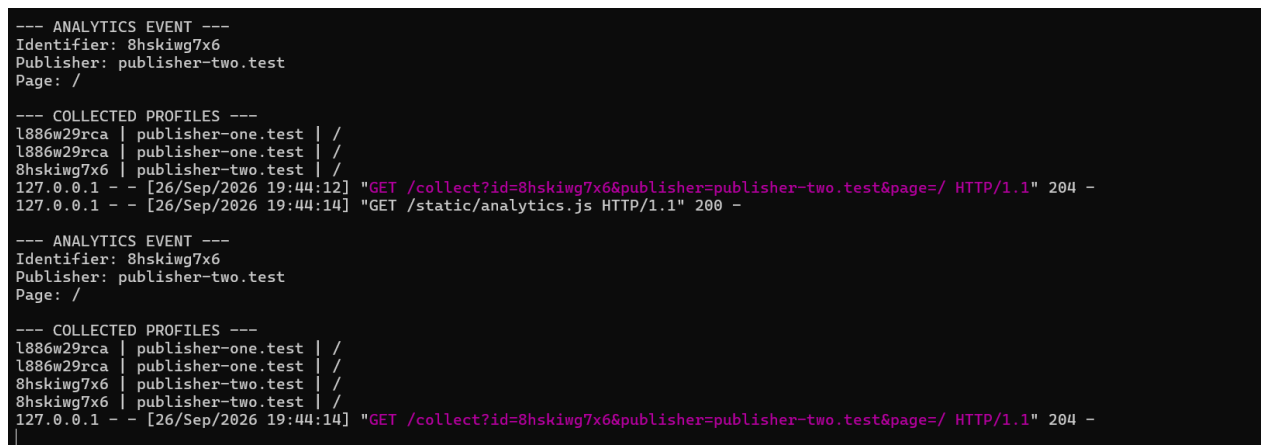
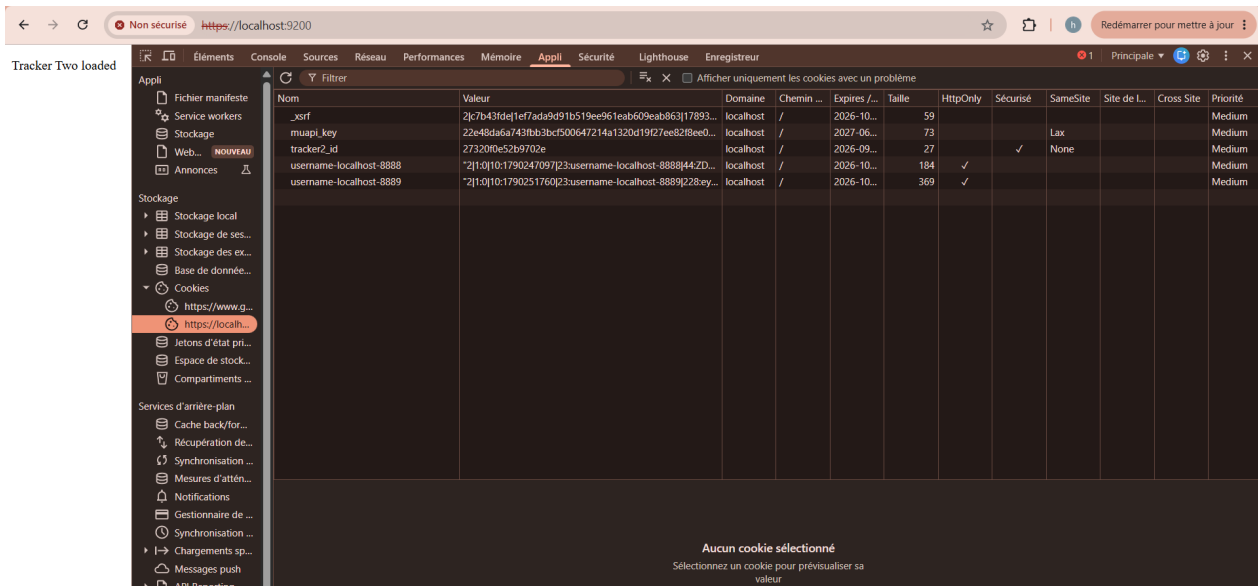


Figure 15: Analytics server collecting two separate first-party profiles.

6 Optional Challenge - Cookie Syncing

A second tracker was created with its own cookie, `tracker2_id`. At this stage, Tracker One and Tracker Two had separate identifiers for the same browser, and neither tracker could directly read the other tracker's cookie.

Figure 16: Independent `tracker2_id` cookie used by Tracker Two.

Cookie syncing was implemented by letting Tracker One send its own identifier to a `/sync` endpoint on Tracker Two. Tracker Two read its own cookie and stored the relationship between the two identifiers. The final mapping was:

`e4c3713d960bf2f3 <-> 27320f0e52b9702e`

```

--- TRACKER TWO REQUEST ---
Tracker 2 ID: 27320f0e52b9702e
127.0.0.1 - - [26/Sep/2026 19:57:18] "GET / HTTP/1.1" 200 -

--- COOKIE SYNC ---
Tracker 1 ID: e4c3713d960bf2f3
Tracker 2 ID: 27320f0e52b9702e

--- SYNCED IDENTIFIERS ---
e4c3713d960bf2f3 <--> 27320f0e52b9702e
127.0.0.1 - - [26/Sep/2026 19:58:13] "GET /sync?tracker1_id=e4c3713d960bf2f3 HTTP/1.1" 200 -

--- COOKIE SYNC ---
Tracker 1 ID: e4c3713d960bf2f3
Tracker 2 ID: 27320f0e52b9702e

--- SYNCED IDENTIFIERS ---
e4c3713d960bf2f3 <--> 27320f0e52b9702e
127.0.0.1 - - [26/Sep/2026 19:58:25] "GET /sync?tracker1_id=e4c3713d960bf2f3 HTTP/1.1" 200 -

```

Figure 17: Successful synchronization of the two tracker identifiers.

7 Conclusion

This lab showed how browser state can be used to recognize a user without an account. A normal cookie can maintain an identifier across requests, while attributes such as `Max-Age`, `HttpOnly`, `SameSite`, and `Secure` change how the cookie is stored and transmitted. Third-party tracking can link activity across different publishers when one tracker identifier is reused. In contrast, first-party analytics creates separate identifiers for different publisher domains unless the service adds another linking mechanism. The optional challenge showed how cookie syncing can connect two identifiers that were initially isolated.